

NYP

Uygulama 5

Daire Sınıfı (return Methodlar)

- Daire adında bir sınıf oluşturun.
- Alan: double yarıcap
- Method: double alanHesapla() $\rightarrow \pi \times r^2$ döndürür (Math.PI kullanın)
- Method: double cevreHesapla() $\rightarrow 2 \times \pi \times r$ döndürür
- Method: String bilgiOzeti() $\rightarrow$ Tüm bilgileri String olarak döndürür
- main'de 2 daire oluşturup hangisinin alanının büyük olduğunu bulun.

NOT: Aynı soru constructor, getter, setter ile derste çözülmüştür.

```
public class Daire {
    public double yaricap;

    public double alanHesapla() {
        return Math.PI * yaricap * yaricap;
    }
    public double cevreHesapla() {
        return 2 * Math.PI * yaricap;
    }
    public String bilgiOzeti() {
        return "Yaricap: " + yaricap
            + " | Alan: " + String.format("%.2f", alanHesapla())
            + " | Cevre: " + String.format("%.2f", cevreHesapla());
    }
}

// ---- Main ----
Daire d1 = new Daire(); d1.yaricap = 5.0;
Daire d2 = new Daire(); d2.yaricap = 3.0;
System.out.println(d1.bilgiOzeti());
System.out.println(d2.bilgiOzeti());
if (d1.alanHesapla() > d2.alanHesapla())
    System.out.println("d1 daha büyük!");
```



Hatırlatma: private Erişim Belirleyici

Encapsulation (Kapsülleme)

Sınıfın iç verilerini dış erişimden korumak için kullanılır.

private anahtar kelimesi ile tanımlanan alanlar (fields)

yalnızca aynı sınıf içinden erişilebilir.

Yanlış Kullanım

```
Ogrenci o = new Ogrenci();  
o.ad = "Ali"; // HATA!  
// ad has private access
```

```
public class Ogrenci {  
    private String ad;  
    private int numara;  
  
    // Bu alanlar sınıf dışından  
    // doğrudan ERİŞİLEMEZ!  
}
```

Doğru Kullanım

```
Ogrenci o = new Ogrenci();  
o.setAd("Ali"); // Doğru!  
String isim = o.getAd();
```

Hatırlatma: Constructor (Yapıcı Metot)

Constructor nedir?

Nesne oluşturulurken otomatik çağrılan özel metottur.

Kuralları:

- Adı sınıf adıyla aynıdır
- Dönüş tipi yoktur (void bile değil)
- Birden fazla constructor olabilir (overloading)
- Yazmazsanız Java varsayılan (default)

constructor sağlar



this anahtar kelimesi, parametre ile field adı aynı olduğunda sınıfın field'ını işaret eder.

```
public class Ogrenci {  
    private String ad;  
    private int numara;  
  
    // Parametrelili Constructor  
    public Ogrenci(String ad, int  
numara) {  
        this.ad = ad;  
        this.numara = numara;  
    }  
  
    // Parametresiz Constructor  
    public Ogrenci() {  
        this.ad = "Bilinmiyor";  
        this.numara = 0;  
    }  
}
```



Hatırlatma: Getter ve Setter Metotlar

Getter (Accessor)

private alana dışarıdan okuma erişimi sağlar.
Dönüş tipi field'ın tipiyle aynıdır.
Parametre almaz.

```
// Getter  
public String getAd() {  
    return this.ad;  
}  
  
public int getNumara() {  
    return this.numara;  
}
```

Setter (Mutator)

private alana dışarıdan yazma erişimi sağlar.
Dönüş tipi void'dir.
Parametre olarak yeni değeri alır.

```
// Setter  
public void setAd(String ad) {  
    this.ad = ad;  
}  
  
public void setNumara(int numara) {  
    this.numara = numara;  
}
```

İsimlendirme Kuralları (Naming Convention)

Eleman	Kural	Örnek
Field (private)	camelCase	<code>studentName, totalPrice</code>
Getter	<code>get + FieldAdı</code>	<code>getStudentName()</code>
Setter	<code>set + FieldAdı</code>	<code>setStudentName(String name)</code>
boolean Getter	<code>is + FieldAdı</code>	<code>isActive(), isVisible()</code>
Constructor	Sınıf adıyla aynı	<code>public Student(...)</code>



boolean tipi field'lar için getter ismi get yerine is ile başlar: `isActive()`, `isVisible()`